SPHERE WEB AUTOMATION

My Courses Script Step-by-Step Walkthrough

A tab-by-tab, control-by-control map of how `tests/Mycourse.spec.ts` verifies the authenticated My Courses page — its shared-tab architecture, its three tabs, and an honest account of which of its 49 tests can actually fail.

Script tests/Mycourse.spec.ts · 541 lines · 49 tests in 7 groups

Target https://sphere.aastrika.org/app/user/my_courses (login required)

Run npm run run:mycourse

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	Authentication and the shared tab	Diagram — four auth modes, one browser tab
3	Page anatomy	Diagram — three tabs, three content shapes
4	Two kinds of test	Diagram — asserting vs observing
5	Group 1 — Page heading and tabs	7 tests
6	Group 2 — Started tab	9 tests
7	Group 3 — For You tab	10 tests
8	Group 4 — Completed tab	6 tests
9	Group 5 — Tab switching	4 tests
10	Group 6 — Header	6 tests
11	Group 7 — Page health	7 tests
12	What green actually proves	18 assert · 31 observe, group by group
13	Names that promise more than the body delivers	Eight tests, and the one-line fixes
14	Artifacts and run variants	Reports, console output, grep recipes
15	Failure modes and fixes	Troubleshooting reference

How to read this document

Sections 5 through 11 walk the script in file order, one table per `describe` group. Each row gives the **action**, the exact **locator**, and what the test does with the result — marked by strength:

Marker	Meaning
assert	An `expect()` always runs. A missing or wrong element fails the test.
conditional	An `expect()` runs only when preconditions hold — typically "only if the account has started courses".
observe	No `expect()` at all. The test measures something and prints it for a human to read.

That last category covers **31 of the 49 tests**, so Section 4 diagrams the two shapes, Section 12 tallies them per group, and Section 13 names the eight tests whose titles read like assertions but are not.

1 What this script does

It is the first suite in the set that runs *inside* the product — the page a logged-in learner lands on to pick up where they left off.

`/app/user/my_courses` is the learner's home base. It carries three tabs: **Started** (courses in progress, each with a progress bar and a Resume button), **For You** (a recommended grid), and **Completed** (which is empty for a new account). This spec walks all three, plus the shared header and a set of page-health checks.

Because the page is behind a login, the suite cannot just navigate to a URL — it needs an authenticated session first. Section 2 covers how it gets one, and the unusual browser-tab arrangement that follows from it.

The file's header promises that "each test is INDEPENDENT — a failure in one doesn't block the others", and that holds: there is no `describe.serial` anywhere, and every test re-navigates through `beforeEach`. What the tests share is a browser *tab*, not state.

The four things it covers

Tab structure

- All three tab labels render with a `(n)` count
- Counts are non-negative integers
- Switching tabs changes the content
- The URL stays on `/my_courses` throughout

Tab contents

- Started: thumbnails, titles, progress, Resume
- For You: card grid, badges, ratings, providers
- Completed: the empty-state message
- Clicking through to a course

Shell and health

- Logo, search bar, Home link, bell, dark mode
- Renders at 390, 768 and 1440px
- Title, reload persistence, console errors
- **Unauthenticated access redirects to login**

> **Its content assertions depend on a real account's real data**
> Several tests look for specific course titles (*Hypertension in Pregnancy*, *Normal Labour*) and providers (*Jhpiego*, *Indian Nursing Council*) that exist only if the logged-in account has those courses. The script handles this by **counting how many of a known list it found and printing the tally** rather than requiring any of them — which is why so much of the suite observes rather than asserts. Section 4 explains the trade; Section 12 measures it.

Entry points

Command	What it runs
`npm run run:mycourse`	All 49 tests, headed, against staging. The full walkthrough this document describes.
`npm run report:app`	This spec alongside Account, CourseTOC and Competency — the authenticated-app blob report.
`… --grep "For You Tab"`	One group. Group names all start with `My Courses –`, so they are unambiguous.

> **Before you run it**
> This suite needs credentials. Without one of the auth modes in Section 2 configured, `authedTest` throws with a message listing what to set — it fails fast rather than running 49 tests against a login page.

2 Authentication and the shared tab

Two architectural decisions shape this file: where the session comes from, and the fact that all 49 tests share one browser tab.

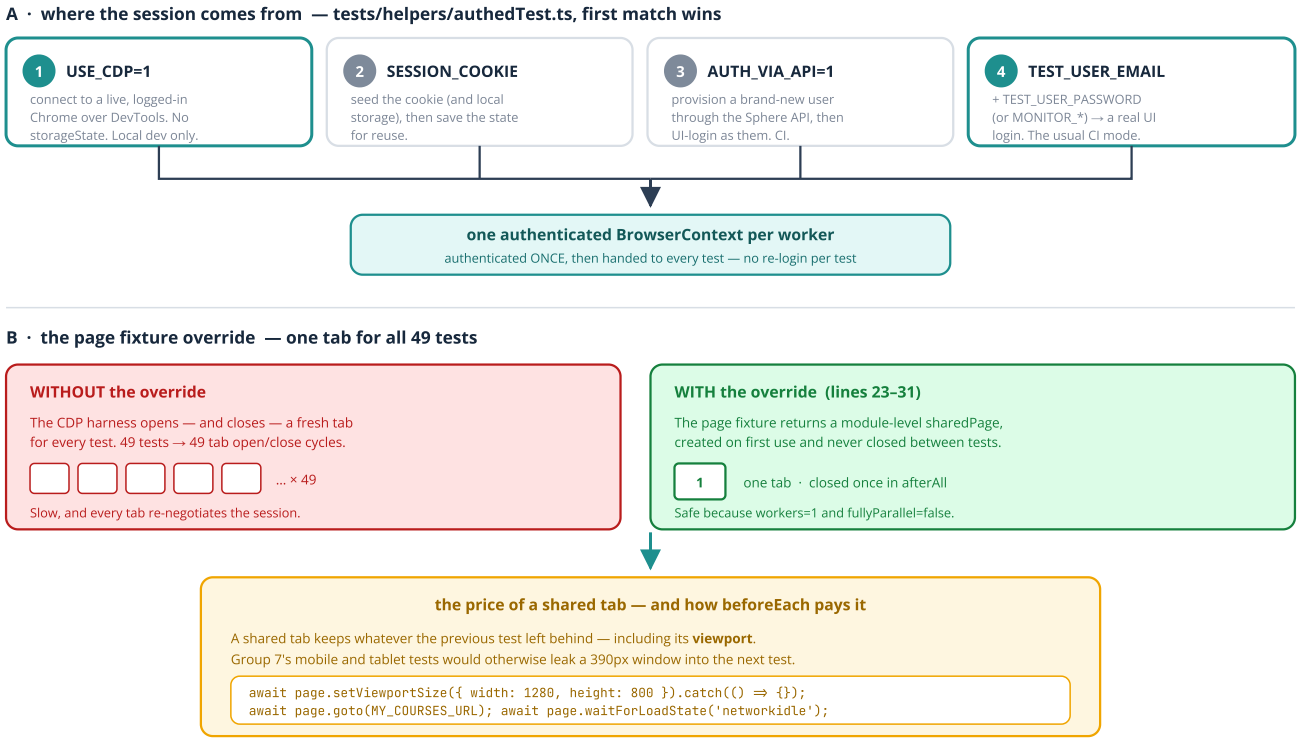


Figure 1 — Authentication is resolved once per worker from whichever env mode is set; the `page` fixture is then overridden so all 49 tests share a single tab, with `beforeEach` resetting viewport and URL to undo any leakage.

The fixture override, in full

```
// :23 — extend the authenticated base test with a shared page
const test = authedBase.extend<{}>({
  page: async ({ context }, use) => {
    if (!sharedPage || sharedPage.isClosed()) {
      sharedPage = await context.newPage();      // created on first use only
    }
    await use(sharedPage);
    // Do NOT close between tests — keep the single tab open.
  },
});

test.afterAll(async () => {
  await sharedPage?.close().catch(() => {});    // closed exactly once
  sharedPage = undefined;
});
```

The `isClosed()` guard matters: if something closes the tab mid-run, the next test transparently opens a new one rather than failing on a dead handle.

> **Why this is safe, and when it would stop being**
> The file's own comment gives the reasoning: `workers=1` plus `fullyParallel=false` means tests already run sequentially, so there is never a second test wanting the tab at the same time. Turn on parallelism and this override breaks immediately — two tests would drive the same tab at once. The shared tab is a deliberate optimisation bought with a config guarantee, not a free one.

3 Page anatomy

One page, three tabs, three completely different content shapes — which is why three of the seven groups are tab-specific.



Figure 2 — The three tabs render three different layouts, which is why Groups 2, 3 and 4 each carry their own `clickTab` hook and their own locators.

4 Two kinds of test

18 of the 49 tests assert. The other 31 measure something and print it. Both shapes appear throughout the file, and telling them apart is the key to reading a run.

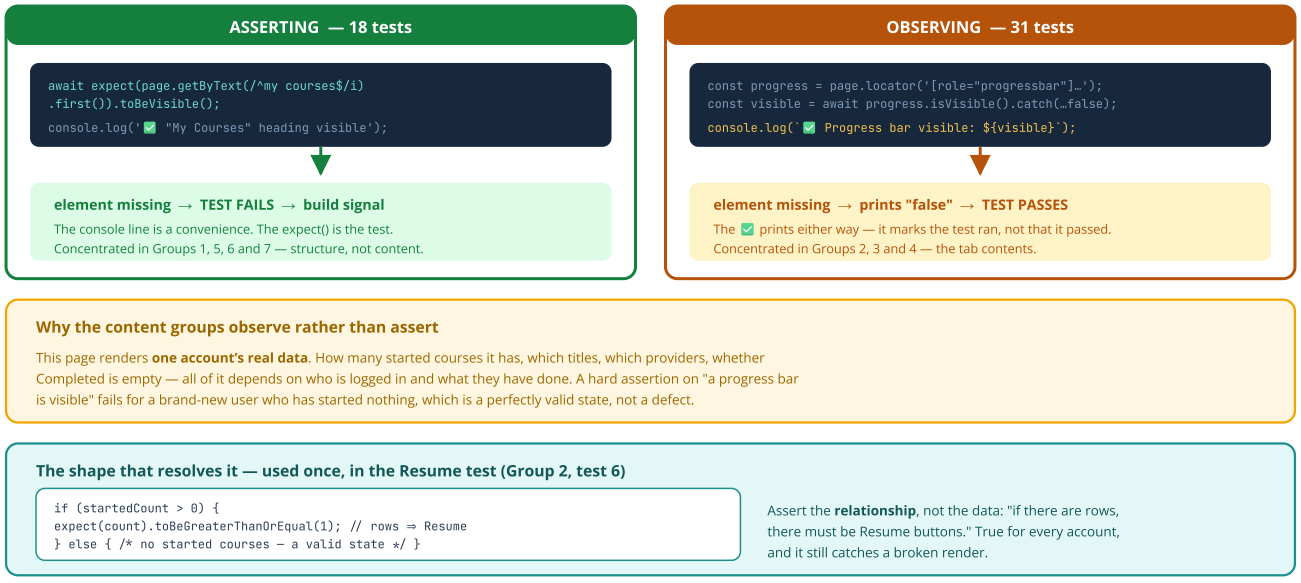


Figure 3 — The two shapes, and the conditional pattern that gets real coverage out of data-dependent content. That pattern appears once in the file; Section 13 lists where else it would apply.

> **The ✅ in the logs does not mean "passed"**
> Almost every test in this file prints a line beginning ✅, including the observing ones — and an observing test prints its checkmark whether the value it measured was `true` or `false`. When reading a run, read the *value* after the colon, not the tick before it.

5 Group 1 — Page heading and tabs

`My Courses — Page Heading & Tabs` · 7 tests · line 79 · 6 of 7 assert

What it proves: the page loaded at the right URL, is correctly titled, and offers all three tabs each carrying a numeric count. This is the most strongly-asserted group in the file, because tab *structure* is the same for every account regardless of data.

#	Action	Locator	Assertion
1	**Page heading**	`getByText(/^my courses$/i).first()`	assert visible. Anchored, so it cannot match the nav link of the same name.
2	**URL is correct**	`page.url()`	assert `toMatch(/my[-_]?courses/i)` — tolerates hyphen or underscore.
3	**"Started" tab with count**	`getByText(/^started\s*\(\d+\)$/i).first()`	assert visible, then the label text is logged. The `\(\d+\)` requires a count to be present.
4	**"For You" tab with count**	`getByText(/^for you\s*\(\d+\)$/i).first()`	assert visible.
5	**"Completed" tab with count**	`getByText(/^completed\s*\(\d+\)$/i).first()`	assert visible.
6	**"Started" is active by default**	`[role="tab"]:has-text("Started"), .mat-mdc-tab:has-text("Started")` → `.first()` then `getAttribute('aria-selected')` and `('class')`	observe — computes `isActive` from `aria-selected === 'true'` or an `/active|selected/` class, then logs both. **No `expect()`.**
7	**Counts are non-negative integers** Loops all three labels.	`new RegExp(`^${name}\s*\(\d+\)$`, 'i')` then `txt.match(/\((\d+)\)/)`	assert ×3 — `toBeGreaterThanOrEqual(0)`. A label whose count fails to parse yields `-1` and fails the test.

The `-1` sentinel in test 7 is doing real work

```
const match = txt?.match(/\((\d+)\)/);
const count = match ? parseInt(match[1]) : -1;   // unparseable → -1
expect(count).toBeGreaterThanOrEqual(0);          // so -1 FAILS
```

Defaulting to `0` instead would have made a missing or malformed count pass silently — the test would assert "0 ≥ 0" and report green on a tab whose label had lost its number entirely. Choosing `-1` turns the absent case into a failure. It is a small decision that determines whether the test is real.

> **Test 6 is the group's one gap**
> It gathers exactly the evidence needed — `aria-selected` and the class list — computes `isActive`, and then prints it instead of asserting it. Which tab opens by default is fixed product behaviour, identical for every account, so it is safely assertable. One line, `expect(isActive).toBe(true)`, would make the test match its title. See Section 13.

6 Group 2 — Started tab

`My Courses — Started Tab` · 9 tests · line 136 · 1 conditional, 8 observe

```
// :137 — every test in this group opens the tab first
test.beforeEach(async ({ page }) ⇒ { await clickTab(page, 'Started'); });
```

What it covers: the in-progress course rows — thumbnail, title, progress bar, percentage text, and the Resume button that takes a learner back into the player.

#	Action	Locator	Assertion
1	**Tab content displayed**	`[class*="course-row"], [class*="course-item"], mat-card`	observe — row count logged.
2	**Thumbnail image**	`img.filter({ hasNotText: 'logo' }).first()`	observe — visibility logged.
3	**Course title** Checks five known titles and counts hits.	`'Initial assessment protocols for ANC', 'Pregnancy Identification', 'Integrated Sexual', 'सम्मानजनक', 'Normal Labour'`	observe — logs `N/5 known titles visible`. Zero hits passes.
4	**Progress bar**	`[role="progressbar"], mat-progress-bar, [class*="progress-bar"]`	observe — visibility logged.
5	**Progress percentage text**	`getByText(/\d+%\s*complete/i).first()`	observe — the matched text logged, or `(none)`.
6	**Resume button per course** The group's only real assertion.	`rows → count, then getByRole('button', { name: /^resume$/i }) → count`	conditional — if rows > 0, `expect(resumeCount).toBeGreaterThanOrEqual(1)`. If zero rows, logs that this is a valid state.
7	**Resume opens the player**	`getByRole('button', { name: /^resume$/i }).first()`	observe — early-returns if absent; otherwise clicks and logs `urlBefore → urlAfter` without comparing them.
8	**Course title click navigates**	`[class*="course"].filter({ hasText: /\w/ }).first()` then its `a, h2, h3, [class*="title"]`	observe — same shape: early return, click, log both URLs.
9	**Tab count matches visible rows**	`tab label \(\(\d+\)\)` vs rows `.count()`	observe — **prints both numbers and never compares them.** See Section 13.

Test 6 is the pattern the rest of the group could adopt

```
// :181 — and the comment explaining why
// A "Resume" appears per in-progress course. An account with no started
// courses legitimately has zero — only require Resume when rows are present.
if (startedCount > 0) {
  expect(count).toBeGreaterThanOrEqual(1);
} else {
  console.log('ℹ️  No in-progress courses on this account — no Resume buttons expected (valid state)');
}
```

This asserts a **relationship between two observations** rather than a fact about the data. It holds for a brand-new account with nothing started and for a heavy user with twelve courses, and it still fails if rows render but their Resume buttons do not. Tests 4, 5 and 9 all have the same structure available — rows exist, therefore progress bars, percentages and matching counts must too.

> **The Devanagari title in test 3**
> `'सम्मानजनक'` sits in the known-titles list alongside four English ones. Course titles are not translated — they are stored as authored — so a Hindi-authored course keeps its own script whatever the interface language. It is the only place in this file that acknowledges non-English content.

7 Group 3 — For You tab

`My Courses — For You Tab` · 10 tests · line 236 · 0 of 10 assert

```
// :237
test.beforeEach(async ({ page }) => { await clickTab(page, 'For You'); });
```

What it covers: the recommendation grid — cards with thumbnails, Certificate badges, star ratings, provider tags and titles, plus the grid layout itself. It is the largest group in the file and the one with no assertions at all.

#	Action	Locator	Assertion
1	**Tab content displayed**	`[class*="course-card"], mat-card`	observe — card count logged.
2	**Cards in a grid**	`cards.filter({ hasText: /Certificate/i })`	observe — certificate-tagged count logged.
3	**Thumbnail on each card**	`cards.first().locator('img').first()`	observe — early-returns if no cards; else logs visibility of the first image only.
4	**"Certificate" badge**	`getByText(/^Certificate$/i).first()`	observe — visibility logged.
5	**Star rating**	`text=/\d\.\d/` → `.first()`	observe — visibility logged. Matches any decimal on the page, not specifically a rating.
6	**Provider tags** Checks four known providers.	`'Indian Nursing Council', 'Jhpiego', 'White Ribbon Alliance', 'UNFPA'`	observe — logs `N/4 known providers visible`.
7	**Course titles** Checks four known titles.	`'Hypertension in Pregnancy', 'Infection Prevention', 'Care of Newborn', 'Newborn Resuscitation'`	observe — logs `N/4 known titles visible`.
8	**Card click opens detail**	`cards.first()`	observe — early return if absent; clicks and logs both URLs.
9	**Tab count matches cards**	tab label vs `cards.count()`	observe — **prints both, never compares.**
10	**3-column grid on desktop** The most interesting test in the group.	Resize to 1440×900, take `boundingBox()` of the first three cards, compare their `y` values	observe — computes `sameRow` and logs it. Skips if fewer than three cards.

Test 10 measures layout geometrically — and then discards the answer

```
const boxes = await Promise.all(cards.slice(0, 3).map((c) => c.boundingBox()));
if (boxes[0] && boxes[1] && boxes[2]) {
  const sameRow = Math.abs(boxes[0].y - boxes[1].y) < 20
               && Math.abs(boxes[1].y - boxes[2].y) < 20;
  console.log(`✅ First 3 cards on same row (3-col grid): ${sameRow}`);
}
```

Comparing the vertical positions of the first three cards, with a 20px tolerance for card-height variation, proves the grid is three-wide at desktop width without hard-coding any CSS. It is the only layout check in the suite that inspects geometry rather than visibility — and the result is printed, not asserted. Column count at 1440px is fixed design, not account data, so `expect(sameRow).toBe(true)` would be safe.

> **Why this group has no assertions — and what could be asserted anyway**

Every element here belongs to a card, and whether any cards exist depends on what the recommendation engine returns for this account. That justifies the caution. But three things in the group are **not** account-dependent: the 3-column layout (test 10), the tab-count-versus-cards relationship (test 9), and the conditional "if cards exist, each must have a thumbnail" (test 3). All three could take Group 2's `if (count > 0)` shape and become real coverage without becoming fragile.

A locator worth tightening

Test 5's `text=/\d\.\d/` matches *any* decimal anywhere on the page — a duration, a price, a version string. On a card grid it almost certainly resolves to a rating, but the locator does not say so. Scoping it to a card would make it mean what its title claims.

8 Group 4 — Completed tab

`My Courses — Completed Tab` · 6 tests · line 340 · 0 of 6 assert

```
// :341
test.beforeEach(async ({ page }) => { await clickTab(page, 'Completed'); });
```

What it covers: the empty state a new learner sees, and the absence of the controls that belong to the other two tabs. Every test here observes — and unlike Group 3, the file explains why in a comment.

#	Action	Locator	Assertion
1	**Tab becomes active**	`[role="tab"]:has-text("Completed"), .mat-mdc-tab:has-text("Completed")` then `aria-selected` + class	observe — both logged.
2	**Count is 0 for a new user**	`tab label → \((\d+)\)`, default `-1`	observe — count logged. The source comment says explicitly: *"Don't assert == 0 strictly — user may have completed courses elsewhere."*
3	**Empty-state message visible**	`getByText(/you have not completed any courses/i).first()`	observe — visibility logged.
4	**Empty-state text matches exactly**	`getByText('You have not completed any courses.', { exact: false }).first()`	observe — if visible, logs the text; else logs that the user may have completed courses. Note `exact: false` despite the title saying "exactly".
5	**No cards when empty**	`cards.filter({ hasText: /Certificate|Resume/i })`	observe — count logged, never required to be zero.
6	**No Resume buttons**	`getByRole('button', { name: /^resume$/i })`	observe — count logged, never required to be zero.

The comment in test 2 is the right call — and it points at a better assertion

Refusing to assert `count === 0` is correct: the suite may run as an account that has genuinely completed something, and failing on that would be wrong. But the group then extends that caution to tests 5 and 6, where it need not apply. Those two are about **internal consistency**, and consistency is assertable whatever the count is:

```
// what tests 5 and 6 measure today — printed only
console.log(`✅ Card count in empty Completed tab: ${count}`);
console.log(`✅ Resume buttons in Completed tab: ${count}`);

// what they could assert, using the count they already read in test 2
if (completedCount === 0) {
  expect(cardCount).toBe(0);      // empty tab ⇒ no cards
  expect(resumeCount).toBe(0);    // Resume belongs to Started, never here
}
```

A Resume button appearing in the Completed tab would be a real defect — it offers to resume a course the learner has finished — and today the suite would print the number and pass.

> **Test 4's title and its code disagree**
> The test is called "Empty-state message text matches exactly" but passes `{ exact: false }` and then logs the text rather than comparing it. What it actually does is a substring search. The empty-state copy is fixed product text, so an exact comparison is entirely safe — this is one of the cleanest one-line fixes in the file.

9 Group 5 — Tab switching

`My Courses — Tab Switching` · 4 tests · line 392 · 2 of 4 assert

What it proves: the tabs are real — switching between them changes what is rendered, and never changes the URL. This is the only group that tests the *transitions* rather than any tab's contents, and it has no nested `beforeEach`: each test drives `clickTab` itself.

#	Action	Mechanism	Assertion
1	**Started → For You shows different content**	Capture `body.innerHTML()` on Started, switch, capture again	assert `expect(startedHTML).not.toEqual(forYouHTML)` — the whole document body must differ.
2	**For You → Completed shows the empty state**	Check the empty-state message before and after the switch	observe — logs `before: false, after: true`. The transition it names is not required to happen.
3	**Completed → Started restores content**	Switch away and back, then look for a Resume button	observe — visibility logged.
4	**URL stays on /my_courses** Loops all three tabs.	`for (const tab of ['Started','For You','Completed'])`	assert ×3 — `toMatch(/my[-_]?courses/i)` after each switch.

Test 1's whole-body comparison

```
const startedHTML = await page.locator('body').innerHTML();
await clickTab(page, 'For You');
const forYouHTML  = await page.locator('body').innerHTML();
expect(startedHTML).not.toEqual(forYouHTML);
```

Comparing the entire `body` is deliberately blunt, and that is its strength: it needs no knowledge of what either tab contains, so it works for any account with any data. If clicking "For You" silently failed — a dead tab, a JavaScript error swallowed by the framework — the two snapshots would be identical and the test would fail.

The weakness is the mirror image. Anything that changes between the two captures satisfies it: a re-rendered timestamp, a rotating banner, a lazily-loaded image that arrived in between. It proves *something* changed, not that the right thing changed. For a tab-switch smoke check that is a reasonable trade, and it is the only tool available that does not depend on the account's data.

> **Tests 2 and 3 are the assertable ones that are not asserted**
> Both describe deterministic product behaviour. Test 2 gathers `beforeEmpty` and `afterEmpty` — the exact pair needed to state "the empty-state message was not on For You and is on Completed" — and prints them. Test 3 reads whether Started's content came back and prints that. Neither depends on how much data the account has, only on whether the tab panel re-rendered, so both could assert. Test 2's assertion would need guarding on the Completed count being zero; test 3's on there being started courses at all — the `if (count > 0)` shape from Group 2 again.

10 Group 6 — Header

`My Courses — Header (same as Home)` · 6 tests · line 433 · 3 of 6 assert

What it proves: the authenticated shell is present and consistent on this page. The group's name states the intent — these are the same controls the Home page carries, re-checked here to catch a header that renders correctly on one route and not another.

#	Element	Locator	Assertion
1	**Aastrika logo**	`img[alt*="aastrika" i], [class*="logo"] img → .first()`	assert visible.
2	**Search bar**	`getByPlaceholder(/what do you want to learn today/i)`	assert visible. Matched by placeholder text — the most stable handle for a search input.
3	**"My Courses" nav link is marked active**	`getByRole('link', { name: /my courses/i }).or(getByText(/my courses/i)).first()` then `getAttribute('class')`	observe — logs the first 100 characters of the class attribute. Whether it contains an active marker is left for a human to judge.
4	**"Home" link navigates** The group's strongest test.	`getByRole('link', { name: /^home$/i }).or(getByText(/^home$/i)).first()`	assert `toMatch(/\/(app|page)\/home/i)` after a 2000 ms wait. This one checks the destination, not just the click.
5	**Dark mode toggle** Six alternative locators.	`mat-slide-toggle · button[aria-label*="theme" i] · button[aria-label*="dark" i] · mat-icon:has-text("dark_mode") · mat-icon:has-text("brightness") · [class*="theme-toggle"]`	observe — visibility logged with an explicit 5 s timeout. The toggle is never operated, only found.
6	**Notification bell**	`mat-icon:has-text("notifications"), [aria-label*="notification" i] → .first()`	observe — visibility logged.

Test 4 is the only navigation in the file with an asserted destination

Eight other tests across the suite click something and log the resulting URL. This one compares it:

```
await scrollAndClick(page, homeLink);
await page.waitForTimeout(2000);
expect(page.url()).toMatch(/\/(app|page)\/home/i);   // both route shapes accepted
```

The alternation is the reason it can assert safely — Sphere serves the authenticated home under both `/app/home` and `/page/home` depending on the build, and accepting either means the test checks "we reached home" without pinning a routing detail. That is the same technique that makes the URL checks in Groups 1, 5 and 7 durable.

> **Test 5's six-way locator is a sign of an unstable control**
> Six alternatives, an explicit 5 s timeout, and an observe-only result together say the dark-mode toggle has moved more than once and is not reliably findable. That is honest — but it also means the test cannot tell "the toggle moved again" from "the toggle was removed". Since it only reports, the practical effect is that a vanished toggle shows up as one `false` in a wall of logs.

11 Group 7 — Page health

`My Courses — Page Health` · 7 tests · line 484 · 6 of 7 assert

What it proves: the page is healthy as a document — renders at every width, keeps its URL across a reload, is correctly titled, and **refuses to serve itself to a visitor who is not logged in**. It contains the file's single most valuable test.

#	Check	Mechanism	Assertion
1	**No console errors on load**	Attach a `console` listener, re-`goto`, wait networkidle + 2000 ms	observe — logs the count and the first five messages, truncated to 200 chars. **No** `expect()`.
2	**Renders on mobile** 390×844	`setViewportSize`, 500 ms settle, then look for the page heading	assert ×1 each — `/^my courses$/i` still visible. These three are why `beforeEach` resets the viewport to 1280×800.
3	**Renders on tablet** 768×1024		
4	**Renders on desktop** 1440×900		
5	**Page title**	`await page.title()`	assert `toMatch(/aastrika|sphere|courses/i)` — three accepted words.
6	**URL persists on reload**	`page.reload()` + networkidle	assert `toMatch(/my[-_]?courses/i)` — proves a refresh does not bounce the user out or lose the route.
7	**Unauthenticated access redirects to login**	`browser.newContext({ storageState: undefined })` → new page → goto → 2000 ms	assert `toMatch(/login|public/i)`, then the context is closed.

Test 7 — the most valuable test in the file

```
// :531 — note the fixture: { browser }, NOT { page }
const context = await browser.newContext({ storageState: undefined });  // no session
const page = await context.newPage();
await page.goto(MY_COURSES_URL);
expect(page.url()).toMatch(/login|public/i);
await context.close();
```

Three things make it stand out. It takes `{ browser }` instead of `{ page }`, deliberately opting out of the shared authenticated tab. It passes `storageState: undefined` to guarantee a sessionless context — inheriting the worker's state would make it pass for the wrong reason. And it closes its own context afterwards.

What it verifies is access control: a logged-out visitor typing the My Courses URL must be bounced to a public route. Everything else in this suite checks presentation; this checks the page is actually protected.

> **Test 1 is one line from being real**
>
> It captures console errors, counts them, prints up to five — and asserts nothing. The sibling `PublicPage.spec.ts` runs the same check and does assert, after filtering resource-load noise. The same two lines work here:
>
> ```
> const jsErrors = errors.filter((e) => !/Failed to load resource/i.test(e));
> expect(jsErrors).toEqual([]);
> ```

> An authenticated page is exactly where an uncaught exception matters most — a broken render here strands a learner mid-course.

12 What green actually proves

18 of 49 tests can fail. The other 31 produce a readable record of the page and always pass.

Group	Tests	Assert	Conditional	Observe	What is actually guaranteed
1 · Heading & Tabs	7	**6**	0	1	Heading, URL, all three tab labels with parseable non-negative counts. Which tab is active by default is only logged.
2 · Started Tab	9	**0**	1	8	Only this: *if* there are course rows, there is at least one Resume button.
3 · For You Tab	10	**0**	0	10	Nothing. Produces a record of cards, badges, ratings, providers, titles and grid layout.
4 · Completed Tab	6	**0**	0	6	Nothing. Records the count, the empty-state message, and the absence of cards and Resume buttons.
5 · Tab Switching	4	**2**	0	2	Started and For You render different content; the URL stays on `/my_courses` through all three tabs.
6 · Header	6	**3**	0	3	Logo, search bar, and that the Home link reaches home. Active-link state, dark toggle and bell are logged.
7 · Page Health	7	**6**	0	1	Renders at three widths; correct title; URL survives reload; **unauthenticated access is blocked**. Console errors are logged only.
Total	**49**	**17**	**1**	**31**	**Structure and access control are gated. Tab contents are recorded, not gated.**

A green run does prove

- The page loads at the right URL for a logged-in user
- All three tabs render with valid counts
- Switching tabs changes content and keeps the route
- The shell works; it renders 390px to 1440px
- **A logged-out visitor is redirected away**

A green run does not prove

- Any course appears in Started or For You
- Progress bars or percentages render
- Cards carry badges, ratings or providers
- The page threw no JavaScript errors

How to read a run

The 31 observing tests emit a structured snapshot of what the page showed — but the information lives in the values, not the pass count. Scan the log for `false` and for zero tallies:

```
✅ Started tab visible rows: 5
✅ First course thumbnail visible: true
   ✓ Title: "Initial assessment protocols for ANC"
✅ 1/5 known titles visible in Started
✅ Progress bar visible: false        ← 5 rows, but no progress bar found
✅ Progress percent text: "(none)"    ← and no percentage either
✅ Resume buttons visible: 5 (started rows: 5)
```

Every line above is green. Together they describe five course rows with Resume buttons intact and **the progress indicator missing** — a real regression, reported as a pass.

The suite's honest value

Treat this file as two things at once: a small, reliable gate on page structure and access control, and a much larger instrument that photographs the page's contents each run. The first protects the build; the second is only useful to someone who reads it.

13 Names that promise more than the body delivers

Eight tests read like assertions in a report. Each one already gathers the value it would need.

These are the tests most likely to mislead. A report listing "*Started tab count matches actual course rows visible ✓*" reads as a verified fact; the body prints two numbers and compares nothing. In every case below the measurement is already taken — only the comparison is missing.

§	Test name	What the body does	The line it is missing
1.6	**"Started" tab is active by default**	Computes `isActive` from aria-selected and class, logs it	`expect(isActive).toBe(true)`
2.9	**Started tab count matches actual course rows visible**	Reads `declaredCount` and `visibleCount`, prints both	`expect(visibleCount).toBe(declaredCount)`
3.9	**For You tab count matches displayed card count**	Reads `declared` and `visible`, prints both	`expect(visible).toBe(declared)`
3.10	**For You cards are arranged in 3-column grid on desktop**	Measures bounding boxes, computes `sameRow`, logs it	`expect(sameRow).toBe(true)`
4.4	**Empty-state message text matches exactly**	Substring-matches with `exact: false`, logs the text	`expect(text?.trim()).toBe(expected)`
4.5	**No course cards are visible when Completed tab is empty**	Counts cards, prints the number	`if (completedCount === 0) expect(count).toBe(0)`
4.6	**No "Resume" buttons appear in Completed tab**	Counts Resume buttons, prints the number	`expect(count).toBe(0)`
5.2	**For You → Completed switch shows empty state**	Reads `beforeEmpty` and `afterEmpty`, prints both	`expect(afterEmpty).toBe(true)`

Which of these are safe to assert, and which need a guard

Safe as-is — fixed product behaviour

- **1.6** Started opens by default for every account
- **3.10** Desktop is three columns by design
- **4.4** The empty-state copy is fixed text
- **4.6** Resume never belongs in Completed

Needs a guard — depends on data

- **2.9 / 3.9** Guard on count > 0 — a paginated or lazily-loaded list may show fewer rows than the tab declares
- **4.5 / 5.2** Guard on the Completed count being zero

Two caveats before changing 2.9 and 3.9

These two look like the most obvious fixes and are the least certain. The tab label's count comes from the API; the visible row count comes from the DOM. They can differ legitimately — if the list virtualises, paginates, or lazy-loads on scroll, fewer rows are in the DOM than the count declares, and a strict equality assertion would fail on a working page. Before asserting equality, confirm the list renders every item at once. If it does not, the honest assertion is `expect(visibleCount).toBeLessThanOrEqual(declaredCount)` plus `toBeGreaterThan(0)` when the declared count is non-zero.

A cheaper fix than changing the assertions

> If these stay as observations, rename them so a report does not overstate what happened — "*Log Started tab count vs visible rows*" rather than "*… matches …*". Only the titles mislead.

14 Artifacts and run variants

14.1 What a run leaves behind

Artifact	Contents
`test-results/`	A screenshot for every one of the 49 tests (`screenshot: 'on'`). Because the tab groups each open their tab first, the set covers all three tab panels plus three viewport widths.
`playwright-report/` · `video.webm`	The HTML report; video kept only for failures.
`console output`	**The primary artifact for this suite.** 31 of the 49 tests put their entire finding here — row counts, title tallies, provider tallies, progress visibility, grid geometry and console-error listings.

14.2 A healthy run, abridged

```
✅ "My Courses" heading visible
   Started: 5 · For You: 12 · Completed: 0
   ✓ Title: "Initial assessment protocols for ANC"
✅ Progress percent text: "11% Complete"
✅ Resume buttons visible: 5 (started rows: 5)
✅ Certificate-tagged cards: 12
✅ 2/4 known For You titles visible
✅ First 3 cards on same row (3-col grid): true
✅ Started vs For You content differs
✅ Home click — URL: …/page/home
✅ Unauth URL: https://sphere.aastrika.org/public/login
```

14.3 Run variants

```
# the whole suite, headed, against staging
npm run run:mycourse

# one group — "Page Health" is the one to triage with
npx playwright test tests/Mycourse.spec.ts --grep "Started Tab"
npx playwright test tests/Mycourse.spec.ts --grep "Page Health"

# auth mode 4 — a real UI login with your own credentials
cross-env TEST_USER_EMAIL=you@yopmail.com TEST_USER_PASSWORD=Sunbird@123 \
  npx playwright test tests/Mycourse.spec.ts --headed

# auth mode 3 — provision a brand-new user through the API first
cross-env AUTH_VIA_API=1 npx playwright test tests/Mycourse.spec.ts

# auth mode 1 — local dev against a live logged-in Chrome over CDP
cross-env USE_CDP=1 npx playwright test tests/Mycourse.spec.ts --headed

# run as the fresh user minted by create-account · or the app blob report
cross-env USE_FRESH_USER=1 npx playwright test tests/Mycourse.spec.ts
npm run report:app
```

Which account you run as changes what the log says

A brand-new user shows `Started (0)`, no Resume buttons and an empty Completed tab — and every observing test still passes, printing zeros. An established account fills the same tests with real values. Neither is a failure, so note which account produced a run you intend to compare against later.

15 Failure modes and fixes

Symptom	Most likely cause	What to do
The run aborts before any test with a credentials error	No auth mode is configured — `authedTest` throws rather than running against a login page.	Set one of the four modes from Section 2. The thrown message lists them. This is the intended behaviour, not a bug.
Every test fails with "element not found" and the URL is /public/login	The session expired or was rejected — the suite is being redirected to login on every navigation.	Delete `.auth/state.json` and re-run so the session is re-established. If mode 1, confirm the CDP Chrome is still logged in.
Tab-specific groups all fail in their hook	`clickTab` could not find the tab — one cause, nine or ten reds in that group.	Check Group 1 first. If the tab labels assert fine there, the markup for the clickable tab changed; extend the five-way selector list at line 50.
A test fails at an unexpected viewport	A previous Group 7 test left the shared tab at 390px and something bypassed the reset.	`beforeEach` resets to 1280×800 — confirm it ran. This is the specific hazard the shared-tab design creates; see Figure 1.
Group 3's grid test logs "Only 2 cards — skipping"	The recommendation engine returned fewer than three cards for this account.	Expected on a sparse account. Run as an established user if you want the grid geometry actually measured.
All the title and provider tallies read 0/5 or 0/4	The account is not enrolled in any of the hard-coded courses, or the catalogue changed.	Not a failure — the tests pass. Update the known-title lists at lines 155 and 277 to match the catalogue you actually test against.
"Started vs For You content differs" fails	The tab click did nothing — the two `body.innerHTML` snapshots came back identical.	A real finding. Confirm with Group 5 test 4, which checks the URL across all three switches.
The unauthenticated-redirect test fails	The page served itself to a sessionless context — a genuine access-control problem.	Treat as an incident, not a test issue. The test builds its own clean context with `storageState: undefined`, so it cannot fail from session leakage.
A test fails on a dead page handle	Something closed the shared tab mid-run.	The fixture's `isClosed()` guard should reopen it transparently. If it recurs, look for a navigation that kills the context — a popup or a hard redirect.
Two tests interfere with each other	Parallelism was enabled, breaking the shared-tab assumption.	This file requires `workers=1` and `fullyParallel=false`. Either restore them or remove the page-fixture override.

> **The one rule to keep**
>
> This page renders one account's real data, so most of its content cannot be asserted absolutely — but almost all of it can be asserted **conditionally**. The Resume test at line 181 shows the shape: measure the precondition, then require the relationship. When you add a test here, take that shape rather than defaulting to a `console.log`, and make sure the test's name describes what its body actually does.